МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

**«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»**

Институт компьютерных технологий и защиты информации

Кафедра Систем Информационной Безопасности

# Лабораторная работа № 3

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Вычисление геометрического центра набора точек: декомпозиция данных и функциональная декомпозиция»

Работу выполнил

Студент гр. 4404

Ионов И.А.

Казань, 2026

## 1. Цель работы

Сравнить два подхода к распараллеливанию: декомпозицию данных (параллельный цикл с редукцией) и функциональную декомпозицию (разные функции в разных потоках).

## 2. Постановка задач

**Задача 5.** Написать программу, которая читает из файла координаты точек (x, y, z) в 3D-пространстве и вычисляет геометрический центр - среднее по x, y и z. Написать две версии: одну с распараллеливанием цикла, другую с функциональной декомпозицией.

## 3. Реализация

Файл lab3/Task5.java; генератор тестовых данных lab3/GenPoints.java; sample.txt - 8 точек с известным ответом (3.25; 4.75; 6.625). Формат файла: одна точка на строку, три числа через пробел.

**Версия A (параллельный цикл):** массив точек делится на блоки по числу потоков; каждый поток накапливает локальные (private) суммы по x, y, z, затем суммы объединяются (редукция, аналог reduction(+:sx,sy,sz)).

**Версия B (функциональная декомпозиция):** три независимые функции в трёх потоках: поток 1 вычисляет среднее по x, поток 2 - по y, поток 3 - по z (аналог #pragma omp sections).

Чтение файла выполняется последовательно и в замер времени вычислений не входит. Результаты версий сверяются с последовательной.

Сборка и запуск (подробнее в README папки lab3):

```
cd lab3
javac Task5.java GenPoints.java
java Task5 sample.txt 2
java GenPoints 5000000 points.txt
java Task5 points.txt 4
```

## 4. Ход работы и скриншоты

**Рабочий код:**

```
import java.io.*;
import java.util.Arrays;

/**
 * Задача 5. Чтение из файла координат точек (x y z) и вычисление геометрического центра
 * (среднее по x, y и z). Две параллельные версии:
 *   A - параллельный цикл (декомпозиция данных): массив делится на блоки, потоки
считают частичные суммы
 *       по x, y, z, затем суммы объединяются (редукция);
 *   B - функциональная декомпозиция: три независимые функции, каждая в своём потоке
 *       (поток 1 суммирует x, поток 2 - y, поток 3 - z). Аналог #pragma omp sections.
 * Для сравнения выполняется последовательная версия.
 *
 * Запуск: java Task5 <файл> [потоки=4]   (потоки используются в версии A)
 * Формат файла: на каждой строке три числа через пробел: x y z
 */
public class Task5 {
    static double[] xs, ys, zs;
    static int n;

    static void readFile(String name) throws IOException {
        int cap = 1024;
        xs = new double[cap]; ys = new double[cap]; zs = new double[cap];
        n = 0;
        try (BufferedReader r = new BufferedReader(new FileReader(name), 1 << 16)) {
            String line;
            int lineNo = 0;
            while ((line = r.readLine()) != null) {
                lineNo++;
                line = line.trim();
                if (line.isEmpty()) continue;
                String[] p = line.split("[\\s;,]+");
                if (p.length < 3) throw new IOException("Line " + lineNo + ": expected 3 numbers");
                if (n == cap) {
                    cap *= 2;
                    xs = Arrays.copyOf(xs, cap); ys = Arrays.copyOf(ys, cap); zs = Arrays.copyOf(zs,
cap);
                }
                xs[n] = Double.parseDouble(p[0]);
                ys[n] = Double.parseDouble(p[1]);
                zs[n] = Double.parseDouble(p[2]);
                n++;
            }
        }
    }

    static double sum(double[] a, int from, int to) {
        double s = 0;
        for (int i = from; i < to; i++) s += a[i];
```

```
        return s;
    }

    static double[] sequential() {
        return new double[]{sum(xs, 0, n) / n, sum(ys, 0, n) / n, sum(zs, 0, n) / n};
    }

    /** Версия A: параллельный цикл с редукцией. */
    static double[] parallelLoop(int p) throws InterruptedException {
        double[][] part = new double[p][3];     // частичные суммы каждого потока (private-копии)
        Thread[] ts = new Thread[p];
        int base = n / p, rem = n % p, from = 0;
        for (int t = 0; t < p; t++) {
            int to = from + base + (t < rem ? 1 : 0);
            final int id = t, f = from, e = to;
            ts[t] = new Thread(() -> {
                double sx = 0, sy = 0, sz = 0;
                for (int i = f; i < e; i++) { sx += xs[i]; sy += ys[i]; sz += zs[i]; }
                part[id][0] = sx; part[id][1] = sy; part[id][2] = sz;
            });
            ts[t].start();
            from = to;
        }
        for (Thread t : ts) t.join();
        double sx = 0, sy = 0, sz = 0;          // reduction(+)
        for (double[] q : part) { sx += q[0]; sy += q[1]; sz += q[2]; }
        return new double[]{sx / n, sy / n, sz / n};
    }

    /** Версия B: функциональная декомпозиция (sections): три функции в трёх потоках. */
    static double[] functional() throws InterruptedException {
        double[] res = new double[3];
        Thread tx = new Thread(() -> res[0] = sum(xs, 0, n) / n);
        Thread ty = new Thread(() -> res[1] = sum(ys, 0, n) / n);
        Thread tz = new Thread(() -> res[2] = sum(zs, 0, n) / n);
        tx.start(); ty.start(); tz.start();
        tx.join(); ty.join(); tz.join();
        return res;
    }

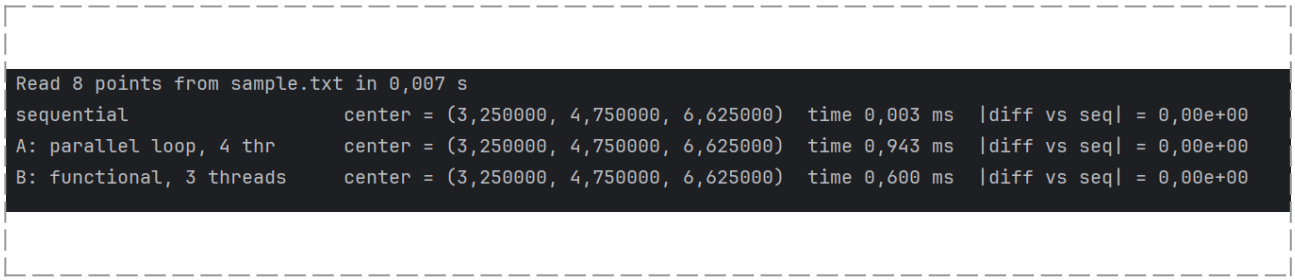
    static void report(String name, double[] c, double[] ref, double sec) {
        double diff = Math.max(Math.abs(c[0] - ref[0]), Math.max(Math.abs(c[1] - ref[1]),
Math.abs(c[2] - ref[2])));
        System.out.printf("%-28s center = (%.6f, %.6f, %.6f)  time %.3f ms  |diff vs seq| = %.2e%n",
                name, c[0], c[1], c[2], sec * 1e3, diff);
    }

    public static void main(String[] args) throws Exception {
        if (args.length < 1) { System.err.println("Usage: java Task5 <file> [threads]"); return; }
        int p = args.length > 1 ? Integer.parseInt(args[1]) : 4;
        if (p < 1) { System.err.println("threads >= 1"); return; }
```

```
        long t0 = System.nanoTime();
        readFile(args[0]);
        double readSec = (System.nanoTime() - t0) / 1e9;
        if (n == 0) { System.err.println("File has no points"); return; }
        System.out.printf("Read %d points from %s in %.3f s%n", n, args[0], readSec);

        // прогрев и измерение: берём лучшее из 5 запусков каждой версии
        double[] ref = null, c;
        double bs = 1e9, ba = 1e9, bb = 1e9;
        double[] ca = null, cb = null;
        for (int r = 0; r < 5; r++) {
            t0 = System.nanoTime(); ref = sequential(); bs = Math.min(bs, (System.nanoTime() - t0) /
1e9);
            t0 = System.nanoTime(); ca = parallelLoop(p); ba = Math.min(ba, (System.nanoTime() - t0)
/ 1e9);
            t0 = System.nanoTime(); cb = functional();    bb = Math.min(bb, (System.nanoTime() - t0) /
1e9);
        }
        report("sequential", ref, ref, bs);
        report("A: parallel loop, " + p + " thr", ca, ref, ba);
        report("B: functional, 3 threads", cb, ref, bb);
    }
}
```

**Скриншот 1. Запуск: `java Task5 sample.txt 2`**




**Пояснение.** Проверка на файле из 8 точек: все версии дают центр (3.25; 4.75; 6.625), совпадающий с расчётом вручную (суммы 26, 38, 53 при N = 8).

## 5. Ответы на вопросы

**Вопрос.** Чем отличаются два подхода и какой масштабируется лучше?

**Ответ.** В версии A работа делится по данным: все потоки выполняют один и тот же код над разными частями массива, число потоков можно менять. В версии B работа делится по функциям: каждый поток выполняет свою функцию (сумма x, y или z), поэтому потоков ровно три, и ускорение не больше трёх. Версия A масштабируется лучше, а версия B проще (не нужны блоки и редукция) и подходит, когда функции различны или независимы.

**Вопрос.** Почему ускорение небольшое?

**Ответ.** Суммирование - операция с малым объёмом вычислений на один элемент, её скорость определяется пропускной способностью памяти. Кроме того, чтение файла (в этой программе последовательное) занимает намного больше времени, чем сами вычисления, поэтому по закону Амдала общее ускорение программы ограничено.

**Вопрос.** Почему результаты версий могут отличаться в последних знаках?

**Ответ.** При разбиении на блоки порядок сложения вещественных чисел меняется, а сложение в формате double не ассоциативно; поэтому расхождение порядка 1e-15 допустимо и не является ошибкой.

## 6. Выводы

- Центр точек вычислен двумя способами; результаты совпадают с последовательной версией и с ручным расчётом на тестовом файле.
- Декомпозиция данных масштабируется по числу потоков, функциональная декомпозиция ограничена числом независимых функций (здесь 3).
- Для задач с лёгкими вычислениями и большим вводом-выводом выигрыш от распараллеливания ограничен последовательной частью программы.